Documentation and Notes

Initial model
3 conv layers
1 fc dense layer
Learning rate=1e-3
Raw SGD

Model was optimized through a series of incremental development, iteration and testing

Learning Rate Optimization

All trials were ran with epochs=20 and batch_size=64

Techniques

Raw SGD
Val acc **41.70%**

Notes
- Does not reach convergence

SGD + Momentum
Val acc (20 epoch) **79.09%**
Val acc (30 epoch) **80.32%**

Notes
- Converges considerably faster than raw SGD
- Plateaus at ~80% accuracy
- Less overfitting than Adam

Adam Optimizer
Val acc **88.15%**

Notes
- Converges faster than SGD + momentum
- Prone to overfitting. Validation loss starts acting erratic after epoch 7

AdamW

Val acc **86.72%**

Notes

- Same problems with regular Adam

Conclusion: I will proceed with SGD + Momentum as the main optimizer

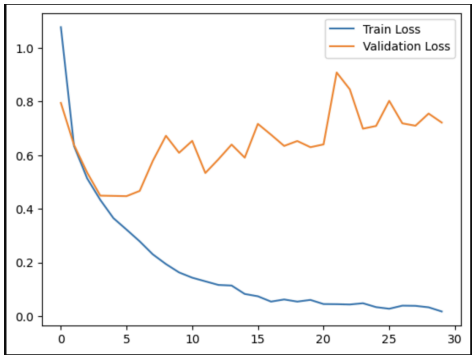
Justification

- Achieves roughly good enough validation accuracy
- Does not overfit as much as both Adam variants
- Converges fast enough for practicality

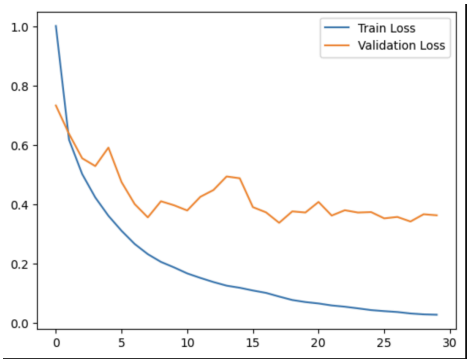
Started with a baseline CNN, diagnosed slow optimization, introduced momentum and BatchNorm, diagnosed a validation plateau and overfitting, then introduced LR scheduling and regularization based on those observations.

Notes: 2:51 am 10/04

Left: Before **BatchNorm**



Right: After **BatchNorm**



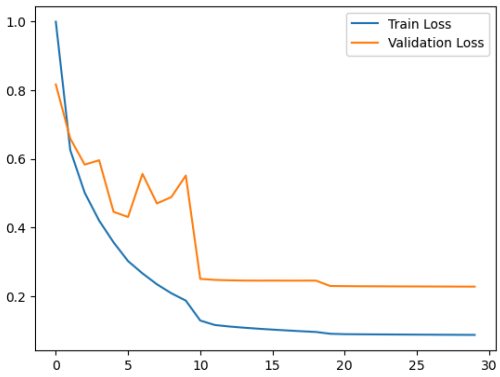
from my observations in the previous hour or so. Definitely

- raw sgd is too slow for any of our purposes, so we need some form of momentum whether that is involved in Adam or SGD + momentum
- batch norm helped the validation curve settle at a lower loss and more stabilized
- Tried L2 regularization and dropout after the dense layer. L2 did not do much, but dropout managed to decrease the train-validation gap slightly

3:14 am - testing out ReduceLROnPlateau

Current stack

- CNN architecture: 3 conv layers + 1 fully-connected linear layer
- SGD + Momentum + BatchNorm + ReduceLR Scheduler + dropout regularization



The above is our loss curve after implementing the ReduceLR scheduler. It substantially improved accuracy from ~85% to 92.05% which allowed us to pass the 90% threshold.

The validation curve also smoothens out at the end instead of getting stuck in an awkward range, preventing us from reaching convergence.

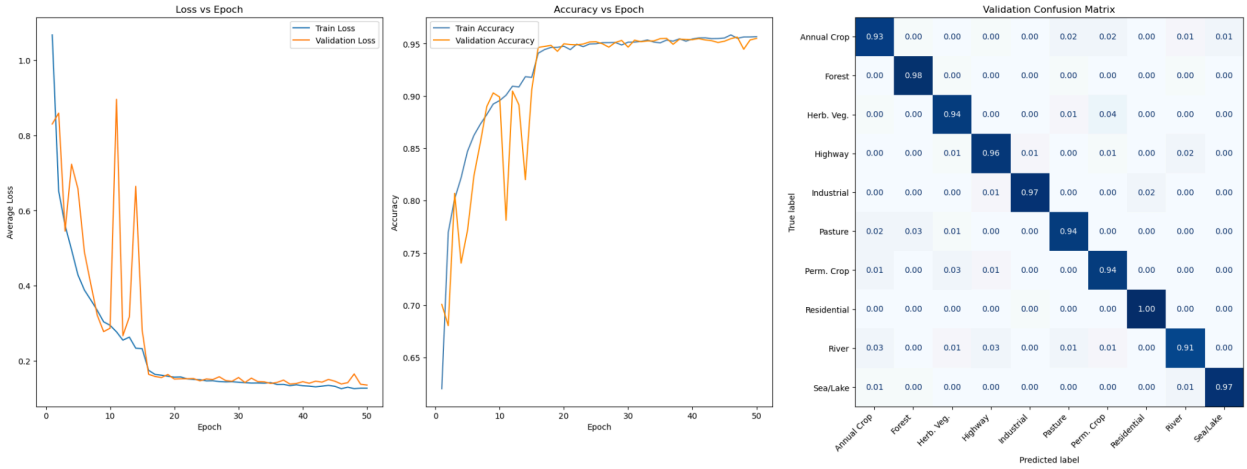
Notes

- The validation curve is approaching closer to the train curve, but there is still a large gap that hints at the lingering overfitting issue.
- Convergence happens at around epoch=20, so we'll shift from doing 30 epochs to only doing 20 as the extra 10 does not add anything meaningful

Final Step: Implement Data Augmentation + cap learning rate reduction to 1e-4

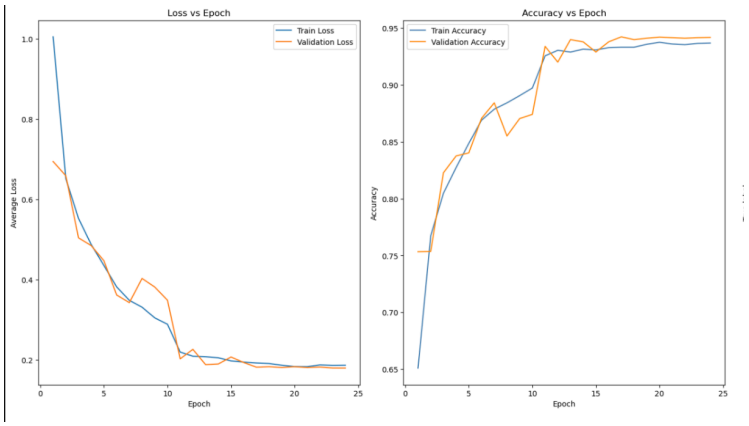
The clear issue is fixing the overfitting problem. The model has problems generalizing to unseen data that was not part of its training set. Data augmentation was introduced to provide variation to the data, such that the model would be seeing new variations in each epoch. This aimed to provide the model with enough chances to learn visual patterns in the data, so it could generalize better.

Final loss curves and confusion matrix (epochs increased to 50 to show convergence)



Side notes:

- I experimented with reducing the learning rate earlier (by reducing the `ReduceLROnPlateau` scheduler's `patience` parameter from 3 to 2)
    - Patience is the number of epochs that the scheduler waits before reducing the learning rate if negligible improvement is seen
- This stabilized the erratic movement from the validation curve at the beginning, but it achieved a lower final accuracy and converged at a much slower rate (which makes sense because the higher learning rate in the beginning allowed the model to make progress faster, albeit more aggressive initially).



This is indeed a cleaner curve, but I decided to finalize the model from before as the best model as it had achieved a higher final accuracy while converging similarly. The tradeoff here is in pre-convergence behavior, but I figured what mattered the most was if the model had converged to good final conditions, which the prior model did just as well.